



AUTOTRIX

Relatório Técnico Final

Plataforma de Agendamento de Serviços Mecânicos

Disciplina: Lab. de Desenvolvimento de Aplicações Móveis e Distribuídas

Curso: Engenharia de Software — PUC Minas

Semestre: 1º Semestre 2026

Aluna: Paloma Dias de Carvalho

Professores: Cleiton Silva Tavares e Cristiano de Macedo Neto

1. Introdução

Este relatório descreve as decisões técnicas, dificuldades encontradas e lições aprendidas ao longo do desenvolvimento do Autotrix, uma plataforma de agendamento de serviços mecânicos construída ao longo de quatro sprints.

O projeto nasceu de uma observação simples: a maioria das oficinas mecânicas ainda gerencia seus agendamentos por telefone ou WhatsApp. Isso cria uma dependência desnecessária de atendimento manual, gera conflitos de horário e deixa o cliente sem visibilidade sobre o andamento do serviço. O Autotrix foi desenhado para resolver exatamente isso, não apenas como exercício acadêmico, mas como uma solução que faz sentido no mundo real.

2. Arquitetura Implementada

O sistema é composto por quatro camadas que se comunicam de formas distintas.

2.1 Backend REST (Flask + PostgreSQL)

O núcleo do sistema é uma API RESTful construída em Flask, orquestrada pelo Docker Compose junto com PostgreSQL, RabbitMQ e um consumer MOM rodando como serviço independente.

A escolha do Flask foi deliberada, visto que ele não impõe estrutura, o backend foi dividido em Blueprints, um por recurso de negócio, cada um responsável por um único domínio: clientes, veículos, serviços, disponibilidades, agendamentos e gestão da oficina. Essa organização reflete diretamente o Princípio da Responsabilidade Única, que Martin (2019, p. 62) define como: cada módulo deve ter um, e apenas um, motivo para mudar.

O banco de dados foi projetado em torno da tabela agendamentos, que conecta clientes, veículos e serviços e controla um ciclo de estados (pendente → confirmado → em_andamento → concluído). Um tipo ENUM nativo no PostgreSQL garante integridade dos estados no nível do banco, não apenas na aplicação.

A autenticação foi implementada de formas diferentes para os dois perfis. Para clientes, usamos bcrypt para hash de senhas, as credenciais nunca trafegam em texto puro. Para a oficina, optamos por credenciais fixas definidas em variáveis de ambiente, o que é suficiente para um único prestador e elimina a complexidade de um sistema de gestão de usuários administrativos.

2.2 Middleware Orientado a Mensagens (RabbitMQ)

A comunicação assíncrona foi implementada com RabbitMQ usando Topic Exchange. Hohpe e Woolf (2003, p. 65) descrevem as quatro opções de integração entre sistemas, File Transfer, Shared Database, Remote Procedure Invocation e Messaging, e concluem que a abordagem por mensagens é a que melhor equilibra desacoplamento e coordenação entre componentes distribuídos.

No Autotrix, quando o cliente cria um agendamento, o backend publica um evento na exchange autotrix.events com a routing key agendamento.criado. Um consumer rodando em container separado consome essa mensagem e a persiste na tabela eventos_log. Como destacam Hohpe

e Woolf (2003, p. 42): "remover as dependências entre os sistemas torna a solução como um todo mais tolerante a mudanças, o principal benefício do acoplamento fraco."

A decisão de usar Topic Exchange em vez de Direct foi pensada para o futuro: se o sistema precisar de um serviço de notificações push, ele pode se inscrever em agendamento.* e receber todos os eventos sem que o producer precise ser alterado.

Quatro eventos compõem o fluxo:

- `agendamento.criado` — notifica a oficina de nova demanda
- `agendamento.status.atualizado` — notifica o cliente de mudança de status
- `agendamento.negociacao.proposta` — oficina propõe novo horário
- `agendamento.negociacao.respondida` — cliente aceita ou recusa

As filas foram declaradas com `durable=True` e as mensagens com `delivery_mode=2`, garantindo persistência em disco. Como ressaltam Coulouris et al. (2011, p. 4), "falhas parciais são uma característica intrínseca dos sistemas distribuídos e devem ser tratadas como casos normais de operação, não como exceções."

2.3 Infraestrutura com Docker Compose

Todos os serviços rodam em containers orquestrados pelo Docker Compose: PostgreSQL, RabbitMQ, backend Flask e consumer MOM. O uso de healthchecks garante que o backend só inicia após o banco e o RabbitMQ estarem prontos, um problema real nas primeiras execuções, quando o backend tentava conectar antes do banco estar disponível.

Os scripts SQL são executados automaticamente na primeira inicialização do banco, na ordem correta graças ao prefixo numérico (`01_init.sql`, `02_sprint2.sql`, `03_sprint3.sql`). Isso garante que qualquer pessoa que clone o repositório consiga rodar o sistema com um único comando.

2.4 Aplicativos Flutter

Foram desenvolvidos dois aplicativos distintos, cada um com identidade visual própria: o app do cliente usa azul (`#1565C0`) e o da oficina usa laranja (`#E65100`).

Ambos seguem a Clean Architecture com três camadas bem definidas: apresentação (screens e widgets), domínio (services e models) e infraestrutura (HTTP e WebSocket). As screens não fazem chamadas HTTP diretamente, elas sempre passam pelos services, que encapsulam a lógica de comunicação.

A atualização em tempo real foi implementada de formas diferentes nos dois apps. No app do cliente, usamos WebSocket: uma conexão persistente que recebe eventos do servidor sem polling. No app da oficina, optamos por polling a cada 10 segundos, mais simples de implementar e suficiente para o caso de uso.

3. Decisões de Design

Por que dois apps separados em vez de um com perfis?

Aplicativos com múltiplos perfis tendem a ter lógica condicional espalhada pelo código, tornando a manutenção mais difícil. Dois apps separados têm responsabilidades claras, interfaces otimizadas para cada perfil e podem evoluir de forma independente.

Por que bcrypt para senhas?

Algoritmos de hash simples como MD5 ou SHA-1 são vulneráveis a ataques de força bruta com tabelas rainbow. O bcrypt é computacionalmente custoso por design, o que dificulta ataques mesmo se o banco for comprometido. Além disso, ele incorpora um salt automático, impedindo que duas senhas iguais gerem o mesmo hash.

Por que soft delete no cancelamento?

Em vez de apagar o registro do agendamento, mudamos o status para cancelado. Isso preserva o histórico, a oficina pode consultar agendamentos cancelados, e o sistema mantém rastreabilidade completa de tudo que aconteceu.

Por que WebSocket para o cliente e polling para a oficina?

O cliente precisa ser notificado imediatamente quando a oficina confirma um agendamento, uma latência de 10 segundos seria perceptível e frustrante. A oficina, por outro lado, consulta uma lista de agendamentos de forma mais passiva. O polling é mais simples de implementar e robusto o suficiente para esse caso.

4. Dificuldades Encontradas e Soluções Adotadas

Como já era esperado, alguns desafios técnicos foram encontrados ao longo do desenvolvimento das Sprints. Abaixo estão as dificuldades mais relevantes e as decisões tomadas para contornar cada situação.

Serialização de tipos PostgreSQL no Flutter

O psycopg2 retorna campos TIME, DATE e NUMERIC como objetos Python nativos (datetime.time, Decimal), que o Flask não serializa para JSON automaticamente. A solução foi fazer o cast diretamente no SQL: hora_inicio::text e preco::text. No Flutter, utilizei double.parse(json['preco'].toString()) em vez de cast direto, que é mais robusto para valores que podem vir como String ou num dependendo do ambiente.

Filas não criadas automaticamente

O RabbitMQ não cria filas quando um producer publica em uma routing key sem binding correspondente, as filas precisam ser declaradas. O consumer declara a infraestrutura ao iniciar, mas nas primeiras execuções ele ainda não estava rodando quando o producer tentou publicar. A solução foi um script de setup que declara filas e bindings manualmente na primeira execução.

Cache corrompido do Docker

Após apagar volumes e fazer rebuild, o Docker apresentou erro de snapshot corrompido que impedia a recriação das imagens. O docker system prune -af limpou o cache completo e resolveu o problema.

Race condition na inicialização dos containers

O backend tentava conectar ao banco antes de ele estar pronto, gerando erros de conexão. A solução foi configurar healthcheck no serviço db e usar condition: service_healthy como dependência do backend.

Endereço do backend no emulador vs navegador

O emulador Android usa 10.0.2.2 para acessar o localhost da máquina host, enquanto o navegador usa localhost diretamente. Isso exige que a URL base nos services seja alterada dependendo de onde o app está rodando, uma limitação operacional que seria resolvida em produção com uma URL fixa.

5. Reflexão sobre os Padrões Estudados

5.1 Event-Driven Architecture (EDA)

Implementar EDA mudou a forma de pensar o sistema. Em vez de pensar em "o cliente chama a oficina", passamos a pensar em "o sistema publica que algo aconteceu, e quem precisar reagir, reage". Isso torna os componentes independentes: o backend não sabe como a notificação chegará ao prestador, e o consumer não sabe como o agendamento foi criado.

O ponto mais interessante foi perceber que a EDA não elimina a necessidade de APIs REST, ela complementa. As operações síncronas (criar agendamento, confirmar horário) continuam sendo REST. A EDA entra para propagar as consequências dessas operações de forma assíncrona.

5.2 Middleware Orientado a Mensagens (MOM)

O RabbitMQ funcionou como um buffer entre o producer e o consumer. Se o consumer estiver temporariamente fora do ar, as mensagens ficam nas filas aguardando, não se perdem. Isso é o que diferencia o MOM de uma chamada HTTP direta: resiliência por desacoplamento.

A configuração de filas duráveis e mensagens persistentes foi uma decisão consciente. Em um ambiente de produção, reinicializações são inevitáveis, e perder notificações de agendamento seria inaceitável.

5.3 Clean Architecture

A separação em camadas foi especialmente útil quando precisamos mudar a URL base do backend para testar no navegador vs emulador. Como toda a comunicação HTTP está encapsulada nos services, a mudança foi em um único lugar, sem tocar nas telas.

O princípio mais valioso da Clean Architecture não é a estrutura de pastas em si, mas a regra de dependência: camadas externas dependem de camadas internas, nunca o contrário. As telas conhecem os services, mas os services não conhecem as telas.

5.4 REST

Usar os verbos HTTP com a semântica correta, especialmente PATCH em vez de PUT para atualizar apenas o status de um agendamento, tornou a API mais expressiva e fácil de entender. Um POST cria, um GET lê, um PATCH altera parcialmente. Isso facilita a vida de quem vai consumir a API no futuro.

6. Conclusão

O Autotrix não é apenas um projeto acadêmico, é uma solução que endereça um problema real e familiar, visto que a ideia surgiu através de situações enfrentadas no empreendimento de um parente. A arquitetura foi elaborada de maneira que poderia ser implantada em produção com

ajustes incrementais. As quatro sprints construíram o sistema de forma progressiva, e cada entrega dependia da anterior.

O maior aprendizado não foi técnico, mas arquitetural: a importância de separar responsabilidades. Um sistema em que cada componente faz uma coisa bem é mais fácil de depurar, de evoluir e de entender. E quando algo dá errado, como invariavelmente acontece, é muito mais fácil encontrar onde.

Referências

COULOURIS, George; DOLLIMORE, Jean; KINDBERG, Tim. **Sistemas distribuídos: conceitos e projeto**. 4. ed. Porto Alegre: Bookman, 2007.

HOHPE, Gregor; WOOLF, Bobby. **Enterprise Integration Patterns: designing, building, and deploying messaging solutions**. Boston: Addison-Wesley, 2003.

MARTIN, Robert C. **Arquitetura limpa: o guia do artesão para estrutura e design de software**. Rio de Janeiro: Alta Books, 2019.